

Onyx — Changes applied to Circle

Onyx · multi-process OS on Raspberry Pi 4

Onyx uses **Circle** ([rsta2/circle](https://github.com/rsta2/circle)) as its hardware-abstraction + driver layer, kept as close to upstream as possible. The few changes we need live in a **fork**, not in the Onyx repository:

- **Fork:** <https://github.com/stephaneweg/circle.git> (remote **origin**), branch **onyx**.
- **Upstream base:** <https://github.com/rsta2/circle.git>, tag **Step51** (remote upstream).
- The **circle/** directory is **not committed** in the Onyx repo — it is gitignored and cloned separately (see the [Developer Guide](#)).

There are **three** patches on top of **Step51**, all small and surgical. To inspect them:

```
git -C circle remote -v # origin = the fork, upstream = rsta2/circle
git -C circle log --oneline Step51..onyx
git -C circle diff Step51..onyx
```

Design rule. Patch Circle as little as possible. When a change can be header-only (no library rebuild), it is. Anything that can live in our own kernel instead (for example the cooked-mouse changed-button handling, done in **kernel1/**'s mouse stub) is **not** a Circle patch.

Some in-source comments still say "Zircon" (the project's former codename) instead of "Onyx" — cosmetic only.

Summary

#	Patch	Files touched	Rebuild needed
1	Runtime keyboard-layout switching · <code>MAX_TASKS 20 → 40</code>	<code>input/keymap.{h,cpp}</code> , <code>usb/usbkeyboard.h</code> , <code>input/keyboardbehaviour.h</code> , <code>sysconfig.h</code>	<code>libinput</code> (+ kernel for <code>MAX_TASKS</code>)
2	<code>CMemorySystem::GetPagerFreeSpace()</code> accessor	<code>memory.h</code>	none (header-only)
3	Heap large-block reuse (fix per-launch canvas leak)	<code>heapallocator.{h,cpp}</code> , <code>sysconfig.h</code>	<code>libcircle</code>

1. Runtime keyboard-layout switching (+ `MAX_TASKS`)

Why. Circle's cooked-mode keyboard loads one country map at boot (`keymap=` on the command line) and offers no way to change it afterwards. Onyx wants **live** layout switching — the `keyb` tool and the theme editor select among US / UK / DE / FR / ES / IT / DV.

What. Add `CKeyMap::LoadMap(locale)`, which re-loads a compiled-in country map over the active one, and expose the keyboard's `CKeyMap` up the device stack so the kernel can reach it.

`lib/input/keymap.cpp` (+ declaration in `include/circle/input/keymap.h`):

```
// switch to another compiled-in country map at runtime; FALSE if the locale is unknown
boolean CKeyMap::LoadMap (const char *pLocale)
{
    const void *pMap = LookupDefaultMap (pLocale);
    if (pMap == 0)
        return FALSE;
    memcpy (m_KeyMap, pMap, sizeof m_KeyMap);
    return TRUE;
}
```

include/circle/input/keyboardbehaviour.h and include/circle/usb/usbkeyboard.h each gain a GetKeyMap() accessor, so the kernel can do CUSBKeyboardDevice::GetKeyMap()->LoadMap("FR"). This backs the kapi_set_keymap / kapi_get_keymap ABI calls (see [Kernel Internals §8](#)).

MAX_TASKS 20 → 40 (include/circle/sysconfig.h): the network stack adds long-lived background tasks (net / DHCP / WPA supplicant + NTP + IRC), so Circle's default of 20 task slots is no longer enough.

2. Pager free-space accessor

Why. The memory-info kapi (the meminfo / memory monitor) needs to report how much of the page-allocator region has not yet been handed out.

What — include/circle/memory.h, **header-only** (no libcircle rebuild):

```
// free bytes of the page-allocator region not yet handed out (freed pages on its
// free list are reused but not counted here).
static size_t GetPagerFreeSpace (void) { return s_pThis->m_Pager.GetFreeSpace (); }
```

3. Heap large-block reuse (fix per-launch leak)

Why. Circle's CHeapAllocator keeps freed blocks on per-size **bucket** free lists, but the buckets only go up to the largest configured size (HEAP_BLOCK_BUCKET_SIZES tops out at 0x80000 = 512 KB). A block **bigger than the top bucket cannot be returned to any free list and is lost** on Free(). Onyx allocates large blocks routinely — a window **canvas** is up to ~3 MB (1024×768×4 bytes) — so **every app launch leaked its canvas**, and the heap was eventually exhausted.

What. Add a set of **power-of-two "large" free lists** alongside the buckets. On Free(), a block too big for any bucket is rounded to its power-of-two size class and pushed onto m_pLargeFreeList[exp]; on Allocate(), a too-big request is rounded up the same way and satisfied from that list first.

include/circle/heapallocator.h:

```
#define HEAP_LARGE_LISTS 32 // per-power-of-2 free lists (cover any realistic size)
...
THeapBlockHeader *m_pLargeFreeList[HEAP_LARGE_LISTS];
```

lib/heapallocator.cpp adds two helpers and wires them into the allocate/free paths:

```
// round a too-big request up to a power-of-two class; return its free-list index
static int HeapLargeClass (size_t *pnSize);
// index of an already-rounded large block (stored size = HEAP_BLOCK_ALIGN << exp)
static int HeapLargeIndex (size_t nRoundedSize);
```

include/circle/sysconfig.h keeps the small bucket list unchanged and documents that larger sizes are now handled by the large lists, so no extra buckets are needed.

Rebuild libcircle after this change (make in circle/lib). Without it, window canvases leak on every launch.

Not a patch: build configuration

One required setting is a **configure option**, not a source change: Onyx renders 32-bit pixels, so Circle must be configured with **DEPTH=32**:

```
cd circle && ./configure -r 4 -p aarch64-none-elf- -d DEPTH=32 -f
```

See the [Developer Guide §2](#) for the full Circle build.

Rebasing the patches onto a newer Circle

```
git -C circle fetch upstream
git -C circle rebase upstream/master onyx      # resolve conflicts in the patched files
# then rebuild the affected libraries:
# - libcircle (heap large-block reuse)
# - libinput  (runtime keymap switching)
# - memory.h  is header-only -> only the kernel that includes it is recompiled
```